

Composing networks from the model zoo

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	Consistent overlap: Ballycanew phosphorus	3
4	Identical up to relabelling: WATER	5
5	Same name, different variable: Rainfall	8
6	Same nodes, inconsistent tables: polar-bear stressor models I and II	10
7	Summary	14
8	References	15

1 Overview

The previous vignette, [Open networks and composition](#), split a network built for the purpose and glued it back together. Published networks are not built to be glued. This vignette takes pairs of them from the model zoo of `EcologicalBayesianNetworks.jl` – files from the Bayesian Network Model Archive ([BNMA 2026](#)), from the `bnlearn` repository ([Scutari 2010](#)) and from the code repository of a paper – and shows what the categorical theory checks when two networks are glued. It checks three things, at three levels:

1. Interface typing. An interface is a `VariableSpace`: variables with their names, their states in order and their space references. `compose`, `glue` and `oapply` compare the two sides before any pushout is taken, and report the first difference as an `InterfaceMismatchError`.
2. The typed-interface rule. Every variable of a composite has at most one mechanism, because a variable on an input foot has none. `Open` refuses an input that has a mechanism (`InterfaceError`), and a composite in which a variable would receive two mechanisms fails validation (`DuplicateGeneratorError`).

3. Kernel consistency. The structure says nothing about the numbers. `merge_kernels` combines the kernel dictionaries of the pieces for `interpret`: it accepts two models that bind the same kernel to a reference, and raises `ConflictingKernelError` when they bind different ones.

Four pairs of zoo models exercise the three checks, one section each: an overlap that is consistent, a network published twice with different labels, one name for two different variables, and two versions of a model that disagree on the nodes they share.

2 Setup

`EcologicalBayesianNetworks` provides the zoo. `model_ir` reads a model file into the `NetworkIR` of `BayesianNetworkFormats.jl`, with the parser options of the model's manifest, and for a model of chance nodes `load_model` builds a `BayesModel` from it: one mechanism per node, called `<node>_mechanism`, with the node's table bound to the reference `NamedRef("<node>_mechanism")`. Two models read from different files therefore share a reference exactly when they share a node name, which is what makes `merge_kernels` a check on them. Each failure below is caught by a small helper that prints the error's message, and that fails itself if the call it is given succeeds.

```
using CategoricalBayesianNetworks
using EcologicalBayesianNetworks
using BayesianNetworkFormats: IRVariable

# Print the message of the error that `f()` raises; every call passed to it below
# fails.
function print_error(f)
    try
        f()
    catch e
        return println(sprint(showerror, e))
    end
    return error("expected an error")
end

zoo = ["ballycanew_diffuse_p", "ballycanew_point_diffuse_p", "water", "bnma_water",
      "native_fish_v1", "waterhole_fence", "polar_bear_stressor_i",
      "polar_bear_stressor_ii"]
[(name, model_info(name).licence, model_info(name).redistribution) for name in zoo]
```

```
8-element Vector{Tuple{String, String, String}}:
 ("ballycanew_diffuse_p", "MIT", "verbatim")
 ("ballycanew_point_diffuse_p", "MIT", "verbatim")
 ("water", "CC BY-SA 3.0", "verbatim")
 ("bnma_water", "CC-BY-SA (version unstated)", "verbatim")
 ("native_fish_v1", "CC-BY (version unstated)", "verbatim")
 ("waterhole_fence", "CC-BY (version unstated)", "verbatim")
 ("polar_bear_stressor_i", "CC-BY-NC-ND (version unstated)", "fetch-only")
```

```
("polar_bear_stressor_ii", "CC-BY-NC-ND (version unstated)", "fetch-only")
```

The last two are fetch-only: the zoo commits a model file only under MIT, CC BY, CC BY-SA or CC0, and keeps just the manifest of a model under a no-derivatives or non-commercial licence. Their section explains what that means here.

3 Consistent overlap: Ballycanew phosphorus

Negri et al. (2024) built two GeNIe models of the phosphorus load of the Ballycanew catchment in Ireland, published under the MIT licence: Model A for diffuse sources (`ballycanew_diffuse_p` in the zoo) and Model B for point and diffuse sources (`ballycanew_point_diffuse_p`). Most of their nodes are GeNIe equation nodes, continuous variables defined by formulas, which the reader cannot represent. Read with `strict = false`, which both manifests set, the reader skips them together with their descendants and keeps the discrete nodes:

```
diffuse = load_model("ballycanew_diffuse_p")
both = load_model("ballycanew_point_diffuse_p")
model_info("ballycanew_point_diffuse_p").parser_options
```

```
Dict{Symbol, Any} with 1 entry:
  :strict => false
```

Model A keeps four variables and Model B twelve: the same four, and eight that describe septic tanks.

```
shared = variable_names(syntax(diffuse))
septic_names = setdiff(variable_names(syntax(both)), shared)
shared, septic_names
```

```
([:month, :Landuse_type, :Morgan_P, :Buffers], [:Ballycanew_ST_risk_classes,
:BallycanewPSaturation100, :leachfield_removal, :Connectivity_rescaled_HSA,
:Directdischarge, :leachfield_connectedness, :ST_Connectedness, :treatment])
```

The four shared variables have the same states, parents and tables in both files, so their kernels are equal:

```
all(kernel(diffuse, x) == kernel(both, x) for x in shared)
```

```
true
```

Both models bind them to the same four references, and `merge_kernels` accepts the pair. Its result is the kernel dictionary of Model B:

```
merged = merge_kernels(diffuse, both)
length(merged), merged == kernels(both)
```

```
(12, true)
```

Nothing in the loaded Model B joins a septic-tank variable to a diffuse one, so Model B should be the monoidal product of Model A and a septic-tank model. The helper piece builds a model from some of the nodes of a file: their mechanisms, with the file's tables bound within the tolerance that `BayesModel(::NetworkIR)` allows for files, and any parent left out as a variable without a mechanism, ready to be an input. The eight septic-tank nodes have no parent outside themselves, so their piece is closed. Tensored with Model A as open networks with empty interfaces, it gives Model B again, with the same kernels:

```
# The part of the network `ir` on the nodes `keep`, as a model with the tables of
# the
# file bound. A parent outside `keep` becomes a variable without a mechanism.
function piece(ir::NetworkIR, keep; atol = 1e-6, renormalize = false)
    nodes = [v for v in ir.variables if v.id in keep]
    outside = [v for v in ir.variables
                if !(v.id in keep) && any(u -> v.id in u.parents, nodes)]
    bn = bayesnet((v.id => Symbol.(v.states) for v in vcat(outside, nodes))...;
                  mechanisms = [v.id => v.parents for v in nodes], closed = false)
    return foldl((m, v) -> bind_cpt(m, v.id => v.table; atol, renormalize), nodes;
                init = BayesModel(bn))
end

both_ir = model_ir("ballycanew_point_diffuse_p")
septic = piece(both_ir, septic_names)
product = otimes(Open(syntax(diffuse)), Open(syntax(septic)))
is_isomorphic(apex(product), syntax(both)), merge_kernels(diffuse, septic) ==
kernels(both)
```

```
(true, true)
```

That is the structural half. The semantic half is that the kernel of Model B is the tensor of the kernels of its two parts. The joint of Model B has more than four million states:

```
prod(nstates(syntax(both), v) for v in variables(syntax(both)))
```

```
4199040
```

Rather than the whole joint, compare the kernels onto the six variables that the skipped equation nodes read: `month`, `Landuse_type`, `Morgan_P` and `Buffers` from the diffuse part, `ST_Connectedness` and `treatment` from the septic-tank part. `interpret` still sums over every state of Model B's apex, which takes a few seconds and needs its limit raised above the default of a million states:

```
from_diffuse = [:month, :Landuse_type, :Morgan_P, :Buffers]
from_septic = [:ST_Connectedness, :treatment]
kA = interpret(Open(syntax(diffuse); outputs = from_diffuse), diffuse)
kS = interpret(Open(syntax(septic); outputs = from_septic), septic)
kB = interpret(Open(syntax(both); outputs = vcat(from_diffuse, from_septic)), both;
```

```

max_states = 5_000_000)
kB ≈ otimes(kA, kS)

```

```
true
```

What the zoo does not load of Model B is what would compose the two sources sequentially. Its equation nodes turn the discrete variables into monthly loads and add the diffuse and point loads into `total_catchment_load`, so the whole tool is the tensor followed by a combining step, $(\text{diffuse} \otimes \text{septic}) \cdot \text{combine}$. The combining step is written entirely in equation nodes, and the non-strict read skips all of them:

```

skipped = [node["id"] for node in both_ir.extras[:skipped]]
length(skipped), "total_catchment_load" in skipped

```

```
(26, true)
```

So the sequential step is not in the networks that the zoo loads: they stop at the tensor, and the two loads are never combined.

4 Identical up to relabelling: WATER

WATER ([Jensen et al. 1989](#)) models a waste-water treatment plant in four time slices of eight variables each, from `_12_00` to `_12_45` by the suffixes of their names. The zoo has it twice: `water`, the HUGIN file of the bnlearn repository ([Scutari 2010](#)), under CC BY-SA 3.0, and `bnma_water`, the BNMA's Netica copy ([BNMA 2026](#)), under CC-BY-SA. The two files list their nodes in different orders, so they are compared node by node:

```

water_ir, bnma_ir = model_ir("water"), model_ir("bnma_water")
byid(ir) = Dict{v.id ⇒ v for v in ir.variables}
wa, wb = byid(water_ir), byid(bnma_ir)
ids = [v.id for v in water_ir.variables]
(same_variables = keys(wa) == keys(wb),
 same_parents = all(wa[x].parents == wb[x].parents for x in ids),
 same_tables = all(wa[x].table == wb[x].table for x in ids),
 same_states = count(wa[x].states == wb[x].states for x in ids))

```

```
(same_variables = true, same_parents = true, same_tables = true, same_states = 0)
```

The same 32 variables, with the same parents in the same order, and all 32 tables equal entry by entry when the states are aligned by position; but not one variable has the same state labels in both. Netica's state names must begin with a letter, so the Netica copy prefixes every label that begins with a digit with N:

```
[(wa[x].states, wb[x].states) for x in (:C_NI_12_00, :CKNI_12_00)]
```

```
2-element Vector{Tuple{Vector{String}, Vector{String}}}:
(["3", "4", "5", "6"], ["N3", "N4", "N5", "N6"])
(["20_MG_L", "30_MG_L", "40_MG_L"], ["N20_MG_L", "N30_MG_L", "N40_MG_L"])
```

(The Netica file does keep the original labels, as the state titles of each node, but the reader does not carry state titles into the NetworkIR.)

To see what the labels do to composition, build WATER from two halves: the first two slices from the bnlearn file, closed, with the eight variables of the second slice as outputs, and the last two slices from the BNMA file, with the same eight variables as inputs (each slice depends only on the one before it).

```
slice(t) = filter(x -> endswith(String(x), t), ids)
first_half = piece(water_ir, vcat(slice("_12_00"), slice("_12_15")))
second_half = piece(bnma_ir, vcat(slice("_12_30"), slice("_12_45")))
A = Open(syntax(first_half); outputs = slice("_12_15"))
B = Open(syntax(second_half); inputs = slice("_12_15"), outputs = slice("_12_45"))
println(A)
println(B)
```

```
OpenBayesNet(16 variables, 16 mechanisms; Symbol[] -> [:C_NI_12_15, :CKNI_12_15,
:CBODD_12_15, :CKND_12_15, :CNOD_12_15, :CBODN_12_15, :CKNN_12_15, :CNON_12_15])
OpenBayesNet(24 variables, 16 mechanisms; [:C_NI_12_15, :CKNI_12_15, :CBODD_12_15,
:CKND_12_15, :CNOD_12_15, :CBODN_12_15, :CKNN_12_15, :CNON_12_15] -> [:C_NI_12_45,
:CKNI_12_45, :CBODD_12_45, :CKND_12_45, :CNOD_12_45, :CBODN_12_45, :CKNN_12_45,
:CNON_12_45])
```

The names line up and the states do not, so the strict sequential composition stops at the first variable of the interface:

```
print_error() do
    compose(A, B)
end
```

```
InterfaceMismatchError: compose: outputs(A) vs inputs(B): interfaces differ in
states at position 1; left has [(Symbol("3"), 1), (Symbol("4"), 2), (Symbol("5"),
3), (Symbol("6"), 4)], right has [(:N3, 1), (:N4, 2), (:N5, 3), (:N6, 4)]
```

The relabelling is a rule: drop an N in front of a digit. It is applied to every node of the BNMA file, and the result must reproduce the bnlearn labels exactly; that check is what makes it a relabelling of the same variables rather than a guess. The helper `relabel` renames one node of a NetworkIR and relabels its states, given as `old ⇒ new` pairs in the new order; listing the old states in another order reorders them, and the tables of the node and of its children are permuted to match.

```
# A copy of node `v` with some of its fields replaced.
function remake(v::IRVariable; id = v.id, states = v.states, parents = v.parents,
               table = v.table)
    return IRVariable(id; v.title, v.kind, states, parents, table, v.deterministic,
```

```

        v.position, v.comment, v.extras)
end

# `ir` with node `id` renamed `name` and its states relabelled by `labels`, a list
# of
# `old ⇒ new` pairs in the new order.
function relabel(ir::NetworkIR, id::Symbol, name::Symbol,
labels::AbstractVector{<:Pair})
    old = only(v.states for v in ir.variables if v.id == id)
    perm = [something(i, 0) for i in indexin(first.(labels), old)]
    length(perm) == length(old) && isperm(perm) ||
        throw(ArgumentError("$first.(labels) does not list the states $old of
$id"))
    along(t, d) = t[ntuple(i -> i == d ? perm : Colon(), ndims(t))...] # permute
axis d
    nodes = map(ir.variables) do v
        t = v.table
        if v.id == id
            return remake(v; id = name, states = last.(labels),
table = t === nothing ? t : along(t, ndims(t)))
        end
        js = findall(==(id), v.parents)
        isempty(js) && return v
        return remake(v; parents = replace(v.parents, id ⇒ name),
table = t === nothing ? t : foldl(along, js; init = t))
    end
    return NetworkIR(ir.name, nodes; ir.format, ir.source, ir.mau, ir.extras)
end

unprefix(s) = length(s) > 1 && s[1] == 'N' && isdigit(s[2]) ? s[2:end] : s
fixed_ir = foldl(bnma_ir.variables; init = bnma_ir) do ir, v
    return relabel(ir, v.id, v.id, [s ⇒ unprefix(s) for s in v.states])
end
fixed = byid(fixed_ir)
all(wa[x].states == fixed[x].states for x in ids)

```

```
true
```

With the second half rebuilt from the relabelled file, the halves compose, and the composite is WATER as the bnlearn file has it:

```

second_half = piece(fixed_ir, vcat(slice("_12_30"), slice("_12_45")))
B = Open(syntax(second_half); inputs = slice("_12_15"), outputs = slice("_12_45"))
water = load_model("water")
is_isomorphic(apex(compose(A, B)), syntax(water))

```

```
true
```

The same holds for the whole networks: the two files give isomorphic networks once the labels are fixed, and not before.

```
bnma = BayesModel(fixed_ir)
(before = is_isomorphic(syntax(water), syntax(BayesModel(bnma_ir))),
after = is_isomorphic(syntax(water), syntax(bnma)))
```

```
(before = false, after = true)
```

The kernels are the third check, and the first attempt to merge them fails:

```
print_error() do
  merge_kernels(water, bnma)
end
```

ConflictingKernelError: two models bind different kernels to BayesianNetworks.NamedRef("CKNI_12_00_mechanism"). `bind\_kernel` derives a mechanism's default reference from its name, so independently built networks that name a mechanism alike collide. Give the mechanisms distinct names before binding their kernels, or set their kernel_ref explicitly. (`rename\_variable` does not rewrite an already-bound reference.)

The tables in the files are equal, but the models were not built alike. The prior of CKNI_12_00 is written as 0.3333333 three times in both files; it is the one row of WATER that does not sum exactly to one, and it misses by 1e-7. The zoo's manifest for water renormalises the tables on loading because of it, and the manifest for bnma_water uses them as read:

```
kernel(water, :CKNI_12_00).table, kernel(bnma, :CKNI_12_00).table
```

```
([0.3333333333333337, 0.3333333333333337, 0.3333333333333337], [0.3333333,
0.3333333, 0.3333333])
```

merge_kernels compares kernels with ==, so a difference in the eighth decimal place is a different kernel. (The advice in the message, to give independently built mechanisms distinct names, is for references that collide by accident; here the two mechanisms are meant to be the same one, and what the error reports is that their kernels are not.) Built with the same option, the two networks bind identical kernels to all 32 references:

```
bnma = BayesModel(fixed_ir; renormalize = true)
length(merge_kernels(water, bnma))
```

32

5 Same name, different variable: Rainfall

Native Fish V1 (BNMA record 107) and Waterhole Fence (record 114) are two small models by Bayesian Intelligence in the BNMA ([BNMA 2026](#)), both under CC-BY, and both have a root node called Rainfall. They share the name and nothing else:

```
fish_ir, fence_ir = model_ir("native_fish_v1"), model_ir("waterhole_fence")
rainfall(ir) = only(v for v in ir.variables if v.id == :Rainfall)
[(v.title, v.states, v.table) for v in rainfall.((fish_ir, fence_ir))]
```

```
2-element Vector{Tuple{String, Vector{String}, Vector{Float64}}}:
 ("Annual Rainfall", ["Below_average", "Average", "Above_average"], [0.1, 0.7, 0.2])
 ("Rainfall", ["Intense", "Low", "Dry"], [0.01369863, 0.05479452, 0.9315069])
```

In Native Fish, Rainfall is annual rainfall against the average, and usually average. In Waterhole Fence it is rainfall intensity, and mostly Dry: its prior is 5, 20 and 340 in 365. Feed the first into the second: A is the Native Fish network with Rainfall as its output, and B is the part of Waterhole Fence between Rainfall and the Fence node – StreamFlow, LandscapeProductivity and HerbivorePressure – with Rainfall as its input and the two chance parents of Fence as its outputs. (Waterhole Fence is an influence diagram; piece takes only those three chance nodes, so its decision and utilities are left out.)

```
fish = load_model("native_fish_v1")
A = Open(syntax(fish); outputs = [:Rainfall])
driven = [:StreamFlow, :LandscapeProductivity, :HerbivorePressure]
B = Open(syntax(piece(fence_ir, driven)); inputs = [:Rainfall],
         outputs = [:StreamFlow, :HerbivorePressure])
print_error() do
    glue(A, B; along = [:Rainfall ⇒ :Rainfall])
end
```

```
InterfaceMismatchError: glue: along[1] = Rainfall => Rainfall: interfaces differ in
states at position 1; left has [(:Below_average, 1), (:Average, 2), (:Above_average,
3)], right has [(:Intense, 1), (:Low, 2), (:Dry, 3)]
```

The typed interface refuses the glue. Both variables have three states, so relabelling them by position would make it go through, and would be wrong: below-average annual rainfall would become Intense, and above-average annual rainfall Dry. The check sees only labels, so once the labels are changed it has nothing to object to:

```
positional = relabel(fence_ir, :Rainfall, :Rainfall,
                    ["Intense" ⇒ "Below_average", "Low" ⇒ "Average",
                     "Dry" ⇒ "Above_average"])
fence_wrong = piece(positional, driven)
G = glue(A, Open(syntax(fence_wrong); inputs = [:Rainfall],
                  outputs = [:StreamFlow, :HerbivorePressure]));
    along = [:Rainfall ⇒ :Rainfall])
```

```
OpenBayesNet(10 variables, 10 mechanisms; Symbol[] -> [:StreamFlow,
:HerbivorePressure])
```

The composite is well typed, and its predictions are not those of Waterhole Fence. The comparison needs Waterhole Fence's own Rainfall prior, which sums to one only within $5e-8$, so that piece is built with `renormalize = true`, as the zoo's manifest for the model does:

```
fence_own = piece(fence_ir, vcat(:Rainfall, driven); renormalize = true)
flow(k) = round.(marginal(k, :StreamFlow).table; digits = 3)
(states = states(syntax(fence_own), :StreamFlow),
  glued = flow(interpret(G, merge_kernels(fish, fence_wrong))),
  waterhole_fence = flow(interpret(Open(syntax(fence_own); outputs = [:StreamFlow]),
    fence_own)))
```

```
(states = [:DestructiveFlood, :Flood, :Dry], glued = [0.08, 0.3, 0.62],
waterhole_fence = [0.011, 0.025, 0.964])
```

The composite expects a destructive flood in 8% of cases and a flood in 30%, where Waterhole Fence itself expects 1.1% and 2.5%. Composing these two networks needs a third piece that neither file contains: a kernel from annual rainfall to rainfall intensity, from someone who knows the catchment. That is the point of typing an interface by its states and not only by its names: here the names agree and the variables do not.

6 Same nodes, inconsistent tables: polar-bear stressor models I and II

Amstrup et al. (2008) forecast the worldwide status of polar bears in the 21st century with a Bayesian network of habitat and other stressors, and Atwood et al. (2016) revised the model. The BNMA holds both versions, Phase I and Phase II (records 146 and 168), which the zoo lists as `polar_bear_stressor_i` and `polar_bear_stressor_ii`. They are licensed CC-BY-NC-ND, so the zoo never commits them or a converted copy: they are fetch-only, and `fetch_model` downloads the original files into the zoo's cache (`cache_dir()`, which the environment variable `ECOLOGICAL_BN_CACHE` can move). Because the licence allows no derivatives, the numbers of these models are not reproduced here: this section prints only names, titles, state labels, true or false verdicts, error messages and counts of nodes, nothing computed from a probability table, and it writes nothing derived from the files.

The section downloads the files, so it runs only when the environment variable `ECOLOGICAL_BN_FETCH` is true, the gate the zoo uses for its own download tests; otherwise it prints a note, and each of the cells after it does nothing.

```
# The zoo's test suite reads its optional gates the same way (`zoo_env_flag`).
zoo_env_flag(name) = lowercase(strip(get(ENV, name, "false"))) in ("true", "1",
"yes")
FETCH = zoo_env_flag("ECOLOGICAL_BN_FETCH")
if !FETCH
    println("Skipped: the polar-bear models are fetch-only (CC-BY-NC-ND). Set ",
        "ECOLOGICAL_BN_FETCH=true to download them into the zoo's cache and run
        ",
        "this section.")
end
```

The node ids differ between the versions (Tourism is J1 in Phase I and Tour in Phase II), so the nodes are matched by title:

```

if FETCH
  records = ("polar_bear_stressor_i", "polar_bear_stressor_ii")
  foreach(r -> fetch_model(r; verbose = false), records)
    irI, irII = model_ir.(records)
    bytitle(ir) = Dict{v.title => v for v in ir.variables}
    tI, tII = bytitle(irI), bytitle(irII)
    titles = sort!(collect(intersect(keys(tI), keys(tII))))
    println(length(irI.variables), " and ", length(irII.variables), " nodes, ",
            length(titles), " titles in common, all of them roots in both: ",
            all(isempty(tI[t].parents) && isempty(tII[t].parents) for t in titles))
    untabled(v) = v.table == nothing ? " (no table)" : ""
    for t in titles
      println(t, "\n I: ", tI[t].id, " ", tI[t].states, untabled(tI[t]),
              "\n II: ", tII[t].id, " ", tII[t].states, untabled(tII[t]))
    end
  end
end

```

38 and 49 nodes, 8 titles in common, all of them roots in both: true

Contaminants

I: T1 ["elevated", "same_as_now", "reduced"]

II: Cont ["elevated", "same as recent", "reduced"]

Ecoregion

I: M ["Polar_Basin_Divergent", "Polar_Basin_Convergent", "Archipelago",
"Seasonal_Ice"]

II: EcoReg ["Archipelago", "Polar_Basin_Convergent", "Polar_Basin_Divergent",
"Seasonal_Ice"]

Hydrocarbons/Oil Spill

I: R4 ["increased_occurrence", "same_as_now", "decreased_occurrence"]

II: OilSpil ["elevated", "same as recent", "reduced"]

Parasites & Disease

I: T ["influential", "not"]

II: ParDis ["elevated", "same as recent", "reduced"] (no table)

Predation

I: T2 ["influential", "not"]

II: Pred ["elevated", "same as recent", "reduced"] (no table)

Shipping

I: J ["increased", "same_as_now"]

II: Ship ["elevated", "same as recent", "reduced"] (no table)

Time Period

I: Q ["historic", "now", "two", "three", "four", "five"] (no table)

II: Per ["historic (1985-1995)", "recent (2007-2012)", "early century
(2020-2030)", "mid-century (2045-2055)", "late century (2070-2080)", "end of
century (2090-2100)"] (no table)

Tourism

I: J1 ["increased", "same_as_now", "decreased"]

II: Tour ["elevated", "same as recent", "reduced"]

Tourism, Hydrocarbons/Oil Spill and Contaminants are the same three-level stressors in both versions, with labels that map one to one (increased, same_as_now, decreased against elevated, same as recent, reduced), and Ecoregion has the same four states in a different order. The other four nodes are redefined rather than relabelled: Shipping goes from two states to three,

Parasites & Disease and Predation from influential and not to three levels, and the six periods of Time Period carry different labels, which only Phase II dates. Phase II gives none of these four a table (they are among the Netica nodes without a CPT that the zoo's manifest lists), and Phase I gives none to Time Period.

Map Phase I onto Phase II's typing explicitly. Every shared node takes its Phase II name; the stressors are relabelled and Ecoregion is reordered; the redefined nodes are only renamed, since there is no relabelling to give them. Then compare the interfaces that the two models offer for each shared variable:

```
if FETCH
  # Phase I's labels of the nodes whose states correspond, listed in the order of
  # Phase II's labels; the redefined nodes keep their own labels.
  labels_I = Dict("Tourism" => ["increased", "same_as_now", "decreased"],
                 "Hydrocarbons/Oil Spill" => ["increased_occurrence",
                                              "same_as_now",
                                              "decreased_occurrence"],
                 "Contaminants" => ["elevated", "same_as_now", "reduced"],
                 "Ecoregion" => ["Archipelago", "Polar_Basin_Convergent",
                                "Polar_Basin_Divergent", "Seasonal_Ice"])
  mapped_ir = foldl(titles; init = irI) do ir, t
    labels = haskey(labels_I, t) ? labels_I[t] .=> tII[t].states :
              tI[t].states .=> tI[t].states
    return relabel(ir, tI[t].id, tII[t].id, labels)
  end
  polarI, polarII = BayesModel(mapped_ir), BayesModel(irII)
  foot(m, x) = output_space(Open(syntax(m); outputs = [x]))
  matches(x) = interface_matches(foot(polarI, x), foot(polarII, x))
  (; (tII[t].id => matches(tII[t].id) for t in titles)...)
end
```

```
(Cont = true, EcoReg = true, OilSpil = true, ParDis = false, Pred = false, Ship =
false, Per = false, Tour = true)
```

The four redefined nodes fail at the interface. oapply glues networks along the junctions of an undirected wiring diagram; with both closed models offering Shipping at one junction, the interfaces are compared before any colimit is taken:

```
if FETCH
  one_junction = @relation () begin
    I(shipping)
    II(shipping)
  end
  print_error() do
    oapply(one_junction, Dict(:I => Open(syntax(polarI); outputs = [:Ship]),
                             :II => Open(syntax(polarII); outputs = [:Ship])))
  end
end
```

InterfaceMismatchError: oapply: junction 1 (box 2, port 2): interfaces differ in states at position 1; left has [(:increased, 1), (:same_as_now, 2)], right has [(:elevated, 1), (Symbol("same as recent"), 2), (:reduced, 3)]

The other four now have the same typing in both models, and the typed-interface rule takes over. They are roots with priors in both versions, so each model has a mechanism for each of them. Feeding Phase I's values into Phase II needs them as Phase II's inputs, and an input has no mechanism:

```
if FETCH
  agreed = [:Tour, :OilSpil, :Cont, :EcoReg]
  print_error() do
    Open(syntax(polarII); inputs = agreed)
  end
end
```

InterfaceError: rule 1 (every input-foot variable has no mechanism in the apex) is violated by variable(s) Tour, OilSpil, Cont, EcoReg (ids 26, 28, 29, 2)

Gluing the two closed models along them instead, each model offering the four as outputs at shared junctions, gives each of them two mechanisms, which validation refuses:

```
if FETCH
  four_junctions = @relation () begin
    I(tourism, oil_spill, contaminants, ecoregion)
    II(tourism, oil_spill, contaminants, ecoregion)
  end
  print_error() do
    oapply(four_junctions, Dict(:I => Open(syntax(polarI); outputs = agreed),
                                :II => Open(syntax(polarII); outputs = agreed)))
  end
end
```

DuplicateGeneratorError: variable :EcoReg (id 9) is the target of 2 mechanisms (ids 9, 40)

The structure says that one of the two models must give up its mechanisms, but not which one. Kernel consistency says whether the choice matters. The mapping named each Phase I mechanism after its Phase II node (Tour_mechanism and so on), so the two models bind kernels to the same four references, and merge_kernels refuses them. Its message names the first conflicting reference it meets, and prints no table:

```
if FETCH
  print_error() do
    merge_kernels(polarI, polarII)
  end
end
```

ConflictingKernelError: two models bind different kernels to BayesianNetworks.NamedRef("Tour_mechanism"). 'bind_kernel' derives a mechanism's default reference from its name, so independently built networks that name a mechanism alike collide. Give the mechanisms distinct names before binding their kernels, or set their kernel_ref explicitly. ('rename_variable' does not rewrite an already-bound reference.)

Node by node, the priors of the three stressors differ between the versions, and EcoRegion's are equal once Phase I's states are reordered:

```
if FETCH
  (; (x ⇒ kernel(polarI, x) == kernel(polarII, x) for x in agreed)...)
end
```

(Tour = false, OilSpil = false, Cont = false, EcoReg = true)

Both versions give EcoRegion a uniform prior:

```
if FETCH
  Tuple(allegal(kernel(m, :EcoReg).table) for m in (polarI, polarII))
end
```

(true, true)

So for EcoRegion either model can generate the variable, the other taking it as an input, and the composite is the same; for the three stressors the versions disagree, and the composite depends on whose prior is kept.

7 Summary

Each pair of networks met a different check.

- **Ballycanew.** Nothing stopped the composition: the four shared variables have identical kernels, so `merge_kernels` accepts the two models, and the twelve-node network is the tensor of the four-node one and a septic-tank model, structurally (`is_isomorphic`, with the same kernels) and semantically (interpret on the six variables that the equation nodes read). The sequential step that combines the two sources lives in equation nodes, which the reader skips.
- **WATER.** Interface typing caught the `N` prefix of the Netica copy (`InterfaceMismatchError`). An explicit relabelling, checked against the other file, let the two halves compose to a network isomorphic to WATER, and made the two files' networks isomorphic. Kernel consistency then caught one prior that the zoo renormalises when it loads one file and not the other; loaded alike, all 32 kernels merge.
- **Rainfall.** Interface typing refused one name for two variables. A positional relabelling would have passed the check and given a composite whose stream floods many times as often.

- Polar bears. Interface typing refused the four redefined nodes; the typed-interface rule refused both ways of gluing the closed models along the other four (InterfaceError for an input with a mechanism, DuplicateGeneratorError for a variable with two); kernel consistency refused the three stressor priors (ConflictingKernelError) and accepted Ecoregion once its states were reordered.

Composing networks that were not built to be composed needs two things that the files do not contain. Between unlike interfaces it needs an explicit map: a relabelling where the states correspond one to one, as for WATER, the polar-bear stressors and the order of Ecoregion, checked against what the files say; and otherwise a kernel from one typing to the other, such as a coarsening of Phase II's three shipping levels into Phase I's two, or a kernel from annual rainfall to rainfall intensity. Where both networks generate a shared variable, it needs a choice of which one generates it, the other taking it as an input; kernel consistency says whether that choice changes the result. The composition operations themselves – compose, glue, otimes, oapply and substitute – are the subject of the previous vignette, [Open networks and composition](#).

8 References

- Amstrup, Steven C., Bruce G. Marcot, and David C. Douglas. 2008. "A Bayesian Network Modeling Approach to Forecasting the 21st Century Worldwide Status of Polar Bears." In *Arctic Sea Ice Decline: Observations, Projections, Mechanisms, and Implications*, edited by Eric T. DeWeaver, Cecilia M. Bitz, and L.-Bruno Tremblay. Geophysical Monograph 180. American Geophysical Union. <https://doi.org/10.1029/180GM14>.
- Atwood, Todd C., Bruce G. Marcot, David C. Douglas, et al. 2016. "Forecasting the Relative Influence of Environmental and Anthropogenic Stressors on Polar Bears." *Ecosphere* 7 (6): e01370. <https://doi.org/10.1002/ecs2.1370>.
- BNMA. 2026. *The Bayesian Network Model Archive*. <https://bnma.co/bnrepo/>.
- Jensen, F. V., U. Kjærulff, K. G. Olesen, and J. Pedersen. 1989. *Et forprojekt til et ekspertsystem for drift af spildevandsrensning (An Expert System for Control of Waste Water Treatment – A Pilot Project)*. Judex Datasystemer A/S.
- Negri, Camilla, Per-Erik Mellander, Nicholas Schurch, et al. 2024. "Bayesian Network Modelling of Phosphorus Pollution in Agricultural Catchments with High-Resolution Data." *Environmental Modelling & Software* 178: 106073. <https://doi.org/10.1016/j.envsoft.2024.106073>.
- Scutari, Marco. 2010. "Learning Bayesian Networks with the bnlearn R Package." *Journal of Statistical Software* 35 (3): 1–22. <https://doi.org/10.18637/jss.v035.i03>.